

SETUP & DEPLOYMENT GUIDE

GG Program Documentation

Installation, Configuration & First-Run Setup Guide

Table of Contents

1. Overview & Technology Stack
2. System Requirements
3. Getting the Source Code
4. Setting Up the Python Environment
5. Environment Variables (Configuration)
6. Database Setup & Migrations
7. Creating the Administrator Account
8. Static Files Setup
9. Running the Development Server
10. Setting Up Automated Backups
11. Project Structure Reference
12. Deploying to Production
13. Updating the Application
14. Setup Troubleshooting

1. Overview & Technology Stack

GG (Green Garden) is a Django web application for managing a cemetery/memorial property business — client records, lot plans, payments, bookings, employees, and automated backups. This guide walks through everything needed to get the program running on a new machine, from installing dependencies to logging in for the first time.

Technology Stack

Component	Technology
Backend framework	Django 6.0.3 (Python)
Database	SQLite (default, file-based — <code>db.sqlite3</code>)
Frontend	Bootstrap 5, vanilla JavaScript, custom CSS
Excel exports	openpyxl
Dependency manager	Pipenv (<code>Pipfile</code> / <code>Pipfile.lock</code>)
Backup delivery	Local ZIP files + optional email (SMTP)

No external database server is required. Green Garden uses SQLite by default, which stores the entire database in a single file — ideal for a single-office or small-business deployment with minimal setup.

2. System Requirements

Requirement	Minimum Version / Notes
Python	3.12 or newer (required by Django 6.0.3)
Pipenv	Latest version (<code>pip install pipenv</code>)
Git	Any recent version, to clone the repository
Operating System	Windows, macOS, or Linux
Disk space	~200 MB for dependencies, plus space for backups and the database
Internet access	Required during setup (to download dependencies) and for email backups

Important: The project's `Pipfile` previously pinned Python 3.14, which does not yet exist as a stable release. Use **Python 3.12** as specified in the `[requires]` section of the `Pipfile` — Django 6.0.3 requires Python $\geq$ 3.12.

3. Getting the Source Code

Option A — Clone with Git

```
git clone --branch test https://github.com/Mito2399/GG.git
cd GG/GG
```

The repository's working application lives in the inner `GG/GG` folder — that is where `manage.py`, `Pipfile`, and the Django project itself are located.

Option B — Download a ZIP

If Git is not available, download the repository as a ZIP from GitHub, extract it, and open a terminal inside the extracted `GG/GG` folder (the one containing `manage.py`).

Verifying You're in the Right Folder

```
ls
# You should see: manage.py, Pipfile, Pipfile.lock, GG/, app/
```

4. Setting Up the Python Environment

4.1 Install Pipenv (if not already installed)

```
pip install pipenv
```

4.2 Install Project Dependencies

From inside the `GG/GG` folder (where `Pipfile` is located), run:

```
pipenv install
```

This reads `Pipfile` / `Pipfile.lock` and installs the exact pinned versions of:

Package	Used for
Django	Core web framework
python-dateutil	Date calculations used across views
openpyxl	Generating Excel (.xlsx) exports

4.3 Activate the Virtual Environment

```
pipenv shell
```

All subsequent commands in this guide (`python manage.py ...`) should be run inside this activated shell.

Alternative: if you prefer not to activate a shell, prefix each command with `pipenv run` , e.g. `pipenv run python manage.py runserver` .

5. Environment Variables (Configuration)

Green Garden reads its configuration from environment variables (see `GG/settings.py`). Set these before first running the server.

Variable	Required?	Default / Notes
ADMIN_PIN	Required	4-digit PIN for the admin account. The server will refuse to start without it.
DJANGO_SECRET_KEY	Recommended for production	Falls back to an insecure default if not set. Always set a unique value before deploying.
DJANGO_DEBUG	Recommended	"True" during development; set to "False" in production.
DJANGO_ALLOWED_HOSTS	Required when DEBUG=False	Comma-separated list of domains/IPs allowed to serve the site, e.g. <code>greengarden.local,192.168.1.10</code> .
BACKUP_OFFLINE_DIR	Optional	Folder where local backup ZIPs are saved. Defaults to a <code>backups/</code> folder inside the project.

5.1 Setting Variables — Windows (PowerShell, persists across sessions)

```
setx ADMIN_PIN "1234"
setx DJANGO_SECRET_KEY "replace-with-a-long-random-string"
setx DJANGO_DEBUG "True"
```

After using `setx`, close and reopen your terminal for the variables to take effect.

5.2 Setting Variables — macOS / Linux (current session only)

```
export ADMIN_PIN="1234"
export DJANGO_SECRET_KEY="replace-with-a-long-random-string"
export DJANGO_DEBUG="True"
```

To make these permanent, add the same lines to your shell profile (`~/.bashrc`, `~/.zshrc`, etc.).

5.3 Using a `.env` File (Alternative)

If your deployment tooling supports it, you may instead place these values in a `.env` file and load them before starting the server. Never commit real secrets (PINs, secret keys, email passwords) to source control.

Choose a real PIN before going live. The example `1234` above is for testing only — pick a non-obvious 4-digit PIN for production use.

6. Database Setup & Migrations

Green Garden uses SQLite by default — no separate database server needs to be installed or configured. The database file (`db.sqlite3`) is created automatically the first time migrations are applied.

6.1 Apply Migrations

```
python manage.py migrate
```

This creates all required tables: clients, beneficiaries, lot plans, payments, bookings, employee logs, activity logs, and system secrets.

What are migrations? Migrations are Django's version-controlled instructions for building and updating the database schema. Always run `migrate` after pulling new code, since new features may add new migration files.

6.2 Verifying the Database

```
python manage.py showmigrations app
```

A checkmark `[x]` next to each migration confirms it has been applied successfully.

7. Creating the Administrator Account

Green Garden's "admin" identity is a special Django superuser. There are two ways to sign in as an administrator:

7.1 Built-in "admin" Username

Any account with the username `admin`, or marked as a Django superuser, is automatically treated as a full Administrator in Green Garden (see `app/context_processors.py`) — no employee record is required for this account.

7.2 Create the Superuser

```
python manage.py createsuperuser
```

Follow the prompts to set a username (commonly `admin`), email (optional), and password.

Admin login uses the password you just set, plus the `ADMIN_PIN` environment variable (not a PIN stored in the database) whenever a PIN confirmation is required — for example, deleting a client or processing other PIN-protected actions.

7.3 Adding Regular Employees

Additional staff accounts (General Staff / Financial Staff) are created from inside the application itself, not the command line — log in as admin, open **Employee → Add Employee**, and fill in the form. See the separate *Green Garden User Tutorial* document for full instructions on day-to-day use.

8. Static Files Setup

Static files (CSS, JavaScript, images) live in `app/static/` during development. Django serves these automatically while `DEBUG=True`.

8.1 Development

No action needed — simply run the server (see Section 9) and static files will be served directly from `app/static/`.

8.2 Production / Collecting Static Files

Before deploying with `DEBUG=False`, collect all static files into the `staticfiles/` folder (configured as `STATIC_ROOT` in `settings.py`):

```
python manage.py collectstatic
```

Type `yes` when prompted to overwrite existing files. Your production web server (e.g., Nginx, IIS, or a WSGI host) should then be configured to serve files from this `staticfiles/` folder directly.

Keep both CSS copies in sync during development. Some stylesheets exist in both `app/static/css/` (source) and `staticfiles/css/` (the generated copy). Always edit the source in `app/static/` and re-run `collectstatic` rather than editing the generated copy directly.

9. Running the Development Server

Step-by-step

1. Make sure your environment variables are set (Section 5) and your virtual environment is activated (`pipenv shell`).
2. Run:

```
python manage.py runserver
```
3. Open a browser and go to: `http://127.0.0.1:8000/`
4. Log in using the superuser credentials created in Section 7.

Running on a Different Port / Network Address

```
# Run on a custom port
python manage.py runserver 8080

# Allow access from other devices on the local network
python manage.py runserver 0.0.0.0:8000
```

When binding to `0.0.0.0` , make sure the device's IP address is included in `DJANGO_ALLOWED_HOSTS` if `DEBUG=False` .

10. Setting Up Automated Backups

Green Garden includes a built-in backup system (`app/backup.py`) that runs inside the Django process — no external scheduler (cron, Task Scheduler) is required.

10.1 How Backups Are Triggered

Trigger	Behavior
Automatic (on data change)	Fires after adding/editing a client, assigning a plan, recording a payment, booking or cancelling an appointment, or managing an employee — subject to a cooldown so rapid changes only produce one backup.
Daily scheduled backup	Runs automatically once per day (default 6:00 PM server time) via a background thread started when the app launches.
Manual ("Backup Now")	Triggered instantly from the Monitor page, or via the command line (below).

10.2 Manual Backup via Command Line

```
python manage.py backup_now
```

This bypasses the cooldown and prints the result (success/failure, file name, and local/cloud delivery status) directly to the console.

10.3 Configuring Where Backups Are Saved

By default, local backups are written to a `backups/` folder inside the project directory. To change this, set the `BACKUP_OFFLINE_DIR` environment variable to an absolute path before starting the server.

10.4 Configuring Email (Cloud) Backups

Email backup credentials are stored **encrypted in the database**, not in source code. Configure them from inside the application:

1. Log in as Administrator.
2. Go to **Monitor → System Settings** (gear icon).
3. Fill in **Email SMTP Host** (e.g. `smtp.gmail.com`), **Port** (e.g. `587`), **Sender Address**, **App Password**, and **Recipient Address**.
4. Enter the current **Admin PIN** and click **Save & Encrypt**.

For Gmail, use an **App Password** (not your normal account password) — generate one from your Google Account's Security settings under "App Passwords."

If email credentials are not configured, the system simply skips the cloud step and keeps the local backup — it will not stop the application from working.

10.5 Backup Retention

Local backups older than **90 days** are automatically deleted after each successful backup, keeping disk usage under control. This can be adjusted in `settings.py` via `BACKUP_RETENTION_DAYS` .

11. Project Structure Reference

```
GG/                                ← project root
└─ GG/                             ← Django project folder (contains manage.py)
    │
    ├── manage.py
    ├── Pipfile / Pipfile.lock
    ├── GG/                         ← project settings package
    │   ├── settings.py             ← all configuration / environment variables
    │   ├── urls.py                 ← top-level URL routing
    │   └── wsgi.py / asgi.py
    ├── app/                        ← main application
    │   ├── models.py               ← database schema (clients, plans, payments, etc.)
    │   ├── views.py                ← page logic / request handling
    │   ├── forms.py                ← form definitions & validation
    │   ├── urls.py                 ← app-level URL routing
    │   ├── admin.py                ← Django admin registrations
    │   ├── middleware.py           ← login rate limiting, admin access control
    │   ├── signals.py              ← auto-backup triggers on data changes
    │   ├── backup.py               ← backup build/save/email logic
    │   ├── crypto.py               ← encryption for stored secrets
    │   ├── apps.py                 ← starts signals + daily backup scheduler
    │   ├── context_processors.py   ← injects role flags into templates
    │   ├── migrations/             ← database schema history
    │   ├── management/commands/backup_now.py
    │   ├── static/                 ← source CSS/JS/images
    │   └── templates/app/          ← all HTML pages
    ├── staticfiles/                ← generated by collectstatic (production)
    ├── backups/                    ← default local backup ZIP location
    └── db.sqlite3                  ← the database file (created after migrate)
```

12. Deploying to Production

12.1 Pre-Deployment Checklist

Item	Action
Secret key	Set a strong, unique <code>DJANGO_SECRET_KEY</code>
Debug mode	Set <code>DJANGO_DEBUG=False</code>
Allowed hosts	Set <code>DJANGO_ALLOWED_HOSTS</code> to your real domain(s)/IP(s)
Admin PIN	Set a unique, non-default <code>ADMIN_PIN</code>
Static files	Run <code>collectstatic</code> and serve <code>staticfiles/</code> via your web server
Database backups	Confirm <code>BACKUP_OFFLINE_DIR</code> points to a durable, backed-up location
HTTPS	Serve the site behind HTTPS in any real deployment

12.2 Running Without the Dev Server

The built-in `runserver` command is for development only. In production, use a production-grade WSGI server such as **waitress** or **gunicorn** pointed at `GG.wsgi.application`, fronted by a reverse proxy (e.g. Nginx or IIS) that also serves the `staticfiles/` folder.

The daily backup scheduler in `app/apps.py` automatically detects whether it is running under `runserver` (dev) or a production server, and starts exactly one background scheduler thread in either case — no extra configuration is required.

12.3 Persisting Environment Variables in Production

Set environment variables through your hosting platform's configuration (systemd service file, Docker environment block, hosting provider's dashboard, etc.) rather than relying on a developer's local `setx` / `export` commands.

13. Updating the Application

Standard update procedure

1. Pull the latest code: `git pull`
2. Activate the environment: `pipenv shell`
3. Install any new dependencies: `pipenv install`
4. Apply new database migrations: `python manage.py migrate`
5. Re-collect static files if any CSS/JS changed: `python manage.py collectstatic`
6. Restart the server (or production process) to load the new code.

Before updating a production system, trigger a manual backup first (`python manage.py backup_now` or the **Backup Now** button) so you have a restore point in case anything goes wrong.

14. Setup Troubleshooting

"RuntimeError: ADMIN_PIN environment variable is not set"

The server refuses to start without this variable. Set `ADMIN_PIN` as shown in Section 5.1/5.2, then restart your terminal session and try again.

"Couldn't import Django. Are you sure it's installed?"

Your virtual environment is not activated, or dependencies were not installed. Run `pipenv install` followed by `pipenv shell` from the folder containing `Pipfile`.

Python version errors during `pipenv install`

Confirm you have **Python 3.12 or newer** installed and that it's the version Pipenv is picking up (`python --version`). The Pipfile's `[requires]` section specifies `python_version = "3.12"`.

Static files (CSS/images) not loading after deployment

Run `python manage.py collectstatic` and confirm your production web server is configured to serve files from the `staticfiles/` folder at the `/static/` URL.

Backup emails are not being sent

Local backups are unaffected — only the email step fails silently if not configured. Go to **Monitor → System Settings** and confirm the SMTP host, port, sender address, app password, and recipient are all filled in correctly. For Gmail, an App Password is required.

"DisallowedHost" error in production

Add your domain or server IP to the `DJANGO_ALLOWED_HOSTS` environment variable (comma-separated, no spaces) and restart the server.

Migrations fail or the database looks out of date

Run `python manage.py showmigrations app` to see which migrations are pending, then run `python manage.py migrate` again. If you're unsure, take a backup of `db.sqlite3` before troubleshooting further.

For day-to-day usage instructions once the program is running, refer to the companion *Green Garden User Tutorial & Documentation*.